PostgreSQL Architecture

A clear, part-by-part explanation of processes, memory, storage and query flow

This guide explains how PostgreSQL is built and how its parts work together. It starts with the big picture and then explains each component in detail: the processes that run, the memory they use, the files stored on disk, how a query is executed, how data stays consistent (MVCC), how it stays safe (WAL), and how it is copied to other servers (replication).

Contents

- 1. The big picture
- 2. Key terminology
- 3. Process architecture
- 4. Memory architecture
- 5. Query processing, step by step
- 6. Storage architecture (files and pages)
- 7. MVCC: how concurrent users are handled
- 8. WAL, checkpoints and crash recovery
- 9. Vacuum and autovacuum
- 10. Locking and concurrency
- 11. Replication, backup and recovery
- 12. System catalogs and statistics
- 13. Configuration and security files
- 14. End-to-end walkthrough: one UPDATE from start to finish
- 15. Summary cheat sheet and commands to explore your own server

1. The big picture

PostgreSQL is an open-source, object-relational database management system. It follows a **client/server model**: applications (clients) send SQL over the network to a server, and the server returns results. Three ideas define its architecture:

- **Process-based design.** PostgreSQL does not use one big multi-threaded program. A supervisor process (the postmaster) starts one separate operating-system process for every client connection, plus several helper processes for background work.
- **Shared memory.** All these processes cooperate through one block of shared memory, which holds the data cache, the write-ahead log buffers and the lock tables.
- **Files on disk.** Data lives in ordinary files inside one directory called the **data directory (PGDATA)**. Changes are first written to a write-ahead log so nothing is lost on a crash.

The diagram below shows all main parts. Read it from top to bottom: a client connects, the postmaster creates a backend for it, and the backend works with shared memory, which in turn is persisted to disk by the background processes.

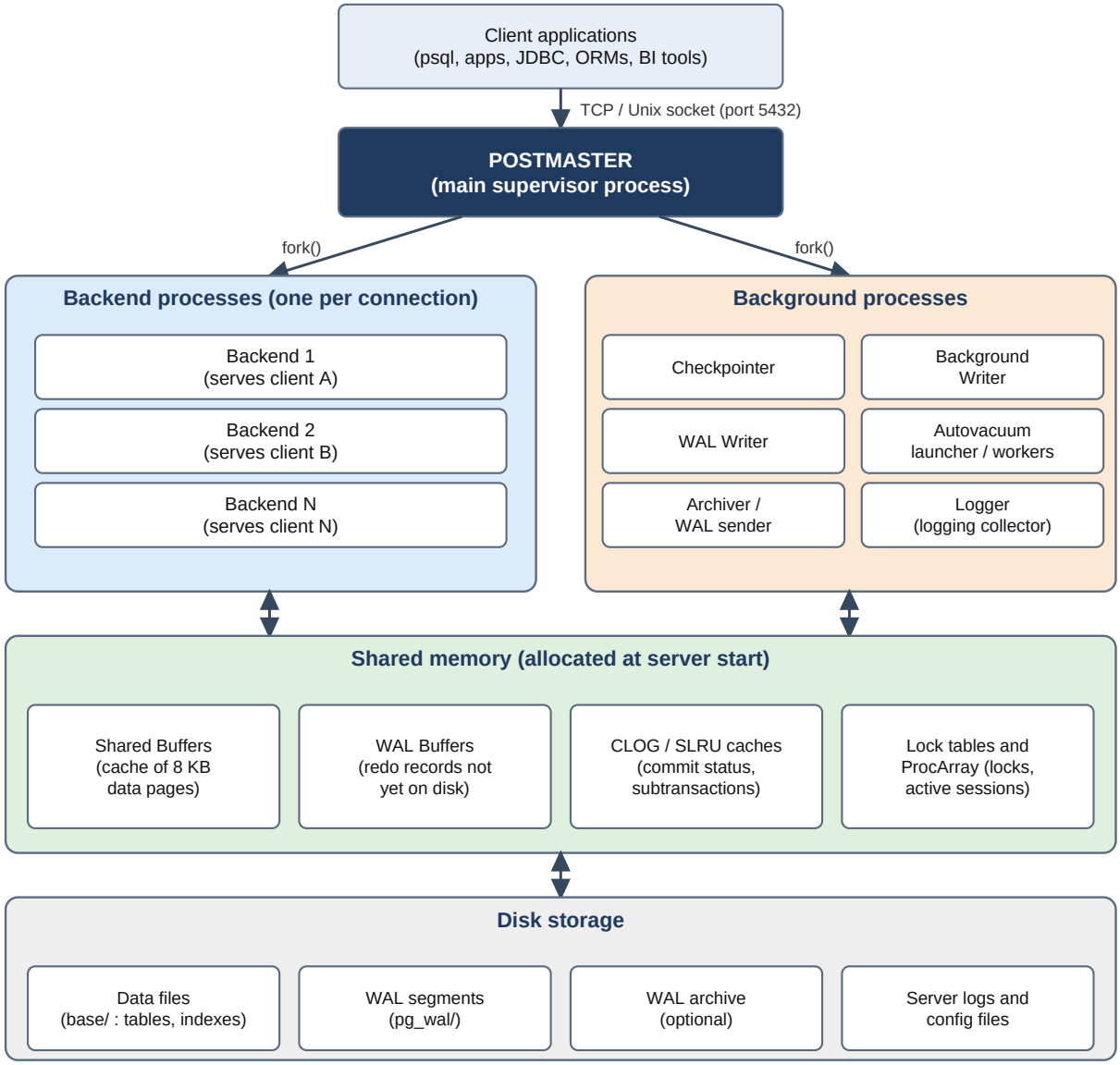


Figure 1: PostgreSQL instance architecture (processes, shared memory and disk)

How to read Figure 1

- **Client applications** connect over TCP/IP or a Unix socket, normally on port 5432.
- **Postmaster** listens for connections and forks a new backend for each one. It also starts and supervises the background processes.
- **Backend processes** do the real work for clients: parse, plan and run SQL.
- **Background processes** do housekeeping that keeps the database fast and safe: writing dirty pages, flushing the log, cleaning dead rows, archiving and logging.
- **Shared memory** is the meeting point; backends read and modify data pages here instead of going to disk each time.
- **Disk storage** holds permanent data files, the write-ahead log (WAL), optional archives and logs.

One running PostgreSQL server (one postmaster, one data directory, one port) is called a **database cluster** or **instance**. It can contain many databases.

2. Key terminology

Term	Meaning
Cluster / instance	One PostgreSQL server with one data directory and one postmaster. Contains many databases. (Not a group of servers.)
Database	A named collection of schemas and objects inside a cluster. A connection works with exactly one database. Defaults: postgres, template0, template1.
Schema	A namespace inside a database that groups tables, views, functions and so on (for example public).
Table / relation	A relation is any table-like object (table, index, view, sequence). A table stores rows.
Tuple	One physical version of a row stored on disk. An UPDATE creates a new tuple version.
Page / block	The fixed 8 KB unit in which PostgreSQL reads and writes table and index data.
Tablespace	A directory location where relation files are stored, letting you place data on specific disks.
OID	Object identifier, the internal number used to identify databases, tables and other objects.
XID	Transaction ID, a 32-bit number given to each transaction that changes data. Used by MVCC.
WAL	Write-Ahead Log: a sequential record of every change, written before the data files change.
LSN	Log Sequence Number: a position in the WAL stream. Pages and checkpoints record LSNs.
Checkpoint	A moment when all changed pages in memory are guaranteed to be written to the data files.
Vacuum	Maintenance that removes dead tuples and updates helper structures and statistics.

3. Process architecture

3.1 The postmaster (main process)

When you start PostgreSQL, the first process that runs is the **postmaster** (the executable is `postgres`). Its jobs are:

- Listen for incoming connections on the configured address and port.
- Start a new **backend process** for each accepted connection (using `fork()`).
- Start the background processes at startup, and restart them if one exits.

- Handle shutdown. *Smart* waits for clients to disconnect, *fast* (default for pg_ctl) disconnects clients and shuts down cleanly with a checkpoint, and *immediate* stops at once and requires crash recovery on next start.
- If any child process crashes abnormally, the postmaster terminates all other backends and runs crash recovery, so shared memory is never left in an unknown state.

The postmaster never runs user queries itself. Its process ID is stored in `postmaster.pid` in the data directory.

3.2 Backend processes

A **backend** (shown as `postgres: user database host state` in `ps`) is created for one client connection and lives until that client disconnects. It:

- Authenticates the client using rules in `pg_hba.conf`.
- Receives SQL, then parses, plans and executes it (see section 5).
- Keeps its own **private memory** (work_mem for sorts and hashes, caches of catalog information).
- Reads and writes data pages through the **shared buffers**, and writes WAL records to the WAL buffers.
- Takes locks and checks MVCC visibility so it sees a consistent snapshot of data.

Because each connection is a full process, very large numbers of connections cost memory and CPU. The limit is `max_connections` (default 100). Applications that open thousands of connections normally use a **connection pooler** such as PgBouncer or pgpool-II.

3.3 Background processes

These helper processes are started by the postmaster. They are the reason PostgreSQL stays fast, durable and tidy without user intervention.

Process	What it does and why it exists	Main settings
Checkpointer	Periodically writes all dirty (changed) pages from shared buffers to the data files and records a checkpoint in the WAL. This limits how much WAL has to be replayed after a crash.	checkpoint_timeout (5 min), max_wal_size (1 GB), checkpoint_completion_target (0.9)
Background Writer (bgwriter)	Continuously writes a small number of dirty pages ahead of time so backends usually find clean buffers available and do not have to write pages themselves.	bgwriter_delay (200 ms), bgwriter_lru_maxpages
WAL Writer	Flushes WAL buffers to the WAL files at regular intervals, so WAL does not pile up in memory and commits are quick.	wal_writer_delay (200 ms)
Autovacuum launcher and workers	The launcher starts worker processes that run VACUUM and ANALYZE on tables that have enough changes. Cleans dead tuples and keeps statistics current.	autovacuum, autovacuum_max_workers (3), autovacuum_naptime (1 min)
Archiver	Copies completed WAL segments to an archive location when archiving is on. Needed for point-in-time recovery.	archive_mode, archive_command
WAL sender / WAL receiver	Used in streaming replication. A walsender on the primary streams WAL to each standby; a walreceiver on the standby receives and stores it.	max_wal_senders, primary_conninfo
Startup process	Runs at server start. Performs crash recovery by replaying WAL, and on a standby keeps replaying WAL continuously.	(automatic)

Process	What it does and why it exists	Main settings
Logger (logging collector)	Collects error and query messages from all processes and writes them to log files.	logging_collector, log_directory
Logical replication launcher / workers	Manage subscriptions for logical replication.	max_logical_replication_workers
Parallel workers	Temporary processes started by a backend to help run one query in parallel.	max_parallel_workers_per_gather

Version notes: before PostgreSQL 15 a separate *stats collector* process gathered usage statistics; since version 15 these are kept in shared memory. Newer versions may also show extra processes such as I/O workers (PostgreSQL 18, asynchronous I/O) or a WAL summarizer (PostgreSQL 17, incremental backup).

3.4 See the processes yourself

```
ps -ef | grep postgres

postgres: checkpointer
postgres: background writer
postgres: walwriter
postgres: autovacuum launcher
postgres: logical replication launcher
postgres: appuser salesdb 10.0.0.5(51234) idle

-- from SQL:
SELECT pid, backend_type, state, usename, datname
FROM pg_stat_activity;
```

4. Memory architecture

PostgreSQL memory falls into two groups: **shared memory**, used by all processes, and **local (private) memory**, used by a single backend.

4.1 Shared memory

Area	Purpose
Shared buffers	The main data cache. Holds copies of 8 KB table and index pages. A backend looks here first; only on a miss is the page read from disk (usually via the operating-system cache). Pages changed in memory are marked **dirty** until written out. Setting: `shared_buffers` (default 128 MB; a common starting point is about 25 percent of RAM on a dedicated server).
WAL buffers	A small area where WAL records are placed before being written to the WAL file. Setting: `wal_buffers` (default is chosen automatically, about 1/32 of shared_buffers, up to 16 MB).
CLOG and other SLRU caches	Small caches for transaction commit status (pg_xact), subtransactions, multixacts and commit timestamps. Used to quickly tell if a transaction committed or aborted.
Lock tables	Information about all heavyweight locks currently held or awaited.
ProcArray	List of running backends and their transaction IDs; used to build MVCC snapshots.
Other	Statistics, replication slot state, background worker slots, and so on.

How buffers are managed: a lookup table maps (relation, page number) to a buffer. When a free buffer is needed, PostgreSQL uses a **clock-sweep** algorithm based on a usage counter, so frequently used pages stay and rarely used pages are replaced. Large sequential scans and vacuum use a small **ring**

buffer so they do not flush the whole cache.

Double caching: PostgreSQL also benefits from the operating-system file cache. So shared_buffers is only part of the story, and `effective_cache_size` is just a hint to the planner about how much total cache (shared buffers plus OS cache) is likely available. It does not allocate memory.

4.2 Local (per-backend) memory

Area	Purpose	Default
work_mem	Memory for each sort, hash table or similar operation inside a query. If an operation needs more, it spills to temporary files on disk. A complex query can use several work_mem chunks at once.	4 MB
maintenance_work_mem	Memory for maintenance tasks: VACUUM, CREATE INDEX, ALTER TABLE ADD FOREIGN KEY.	64 MB
temp_buffers	Per-session buffers for temporary tables.	8 MB
Catalog and plan caches	Each backend caches table definitions and prepared plans it has used.	(dynamic)

Sizing warning: work_mem is used per operation, per query, per connection. With 200 connections each running a query that has 3 sort nodes at 64 MB, the potential total is 200 x 3 x 64 MB, far more than you would expect. Increase work_mem carefully.

5. Query processing, step by step

When a backend receives a SQL statement it passes it through a fixed pipeline. Figure 2 shows the stages.

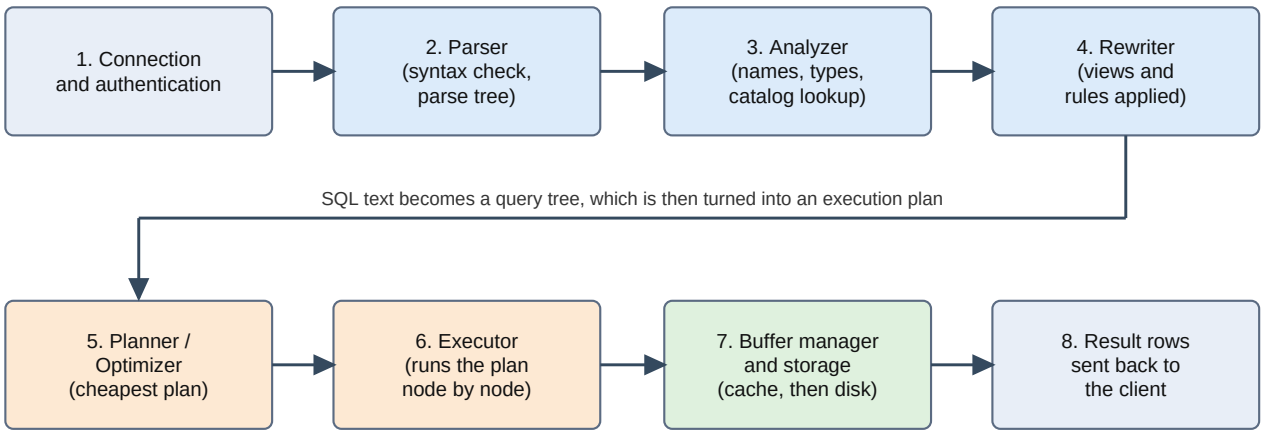


Figure 2: Life of a query inside a backend process

Stage	What happens in detail
1. Connection and authentication	The client connects; the postmaster forks a backend. The backend checks `pg_hba.conf` (which hosts, users and databases are allowed and by which method, such as scram-sha-256) and verifies the password or certificate.
2. Parser	The SQL text is checked for correct syntax and converted into a **parse tree**. A syntax error (for example a misspelled keyword) is reported here.
3. Analyzer (semantic analysis)	Table and column names are looked up in the system catalogs, data types are resolved and permissions are checked. The result is a **query tree**. Errors such as 'relation does not exist' appear here.

Stage	What happens in detail
4. Rewriter	Applies rules and expands views. For example, a query on a view is rewritten into a query on the underlying tables.
5. Planner / Optimizer	Generates many possible ways to run the query and picks the one with the lowest estimated **cost**, using table statistics gathered by ANALYZE. It chooses scan methods (sequential, index, bitmap), join order and join methods (nested loop, hash join, merge join), and decides whether to use parallel workers.
6. Executor	Runs the chosen plan as a tree of nodes. Each node asks its child nodes for rows one at a time (iterator model), for example Sort pulling rows from a Hash Join, which pulls rows from two scans.
7. Buffer manager and storage	Scan nodes request pages. The buffer manager returns them from shared buffers or reads them from disk. MVCC visibility rules decide which row versions the query may see. Changes also produce WAL records.
8. Result	Rows are returned to the client as they are produced. Locks are released at the end of the transaction.

Seeing the plan

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.name, sum(o.total)
FROM customers c JOIN orders o ON o.customer_id = c.id
WHERE o.created_at >= now() - interval '7 days'
GROUP BY c.name;
```

EXPLAIN shows the plan the planner chose. With ANALYZE it also runs the query and reports actual times and row counts; BUFFERS shows how many pages came from shared buffers (hits) versus disk (reads). Comparing estimated and actual row counts is the first step in diagnosing a bad plan.

Planner statistics: the planner relies on the catalog `pg_statistic` (viewable as `pg_stats`), filled by ANALYZE. It holds row counts, most common values, histograms and correlation. Stale statistics lead to poor plans, which is why autovacuum also runs ANALYZE.

6. Storage architecture (files and pages)

6.1 Logical hierarchy

Inside a cluster the structure is: **cluster -> databases -> schemas -> objects** (tables, indexes, views, functions, sequences). Roles (users) and tablespaces are defined at cluster level and are shared by all databases.

6.2 The data directory (PGDATA)

Everything physical lives under one directory. Find it with `SHOW data_directory;`. Its main contents are:

Path	Contents
base/	One subdirectory per database (named by database OID). Contains the files of every table and index in that database.
global/	Cluster-wide tables (such as the list of databases and roles) and the **pg_control** file, which records the latest checkpoint location and cluster state.
pg_wal/	The write-ahead log, as 16 MB segment files (formerly called pg_xlog).
pg_xact/	Transaction commit status (the CLOG; formerly pg_clog).

Path	Contents
pg_multixact/, pg_subtrans/, pg_commit_ts/	Multi-transaction row locks, subtransaction parents and commit timestamps.
pg_tblspc/	Symbolic links to tablespace locations.
pg_stat/, pg_stat_tmp/	Saved and temporary statistics data.
pg_replslot/, pg_logical/	Replication slot state and logical decoding data.
pg_twophase/	State for prepared (two-phase commit) transactions.
PG_VERSION	The major version of the cluster.
postgresql.conf, postgresql.auto.conf	Main configuration, and settings written by ALTER SYSTEM.
pg_hba.conf, pg_ident.conf	Client authentication rules and user-name mapping.
postmaster.pid	Lock file with the postmaster process ID. Exists only while running.

On Debian and Ubuntu the configuration files are usually kept in `/etc/postgresql/<version>/main/` instead of inside PGDATA.

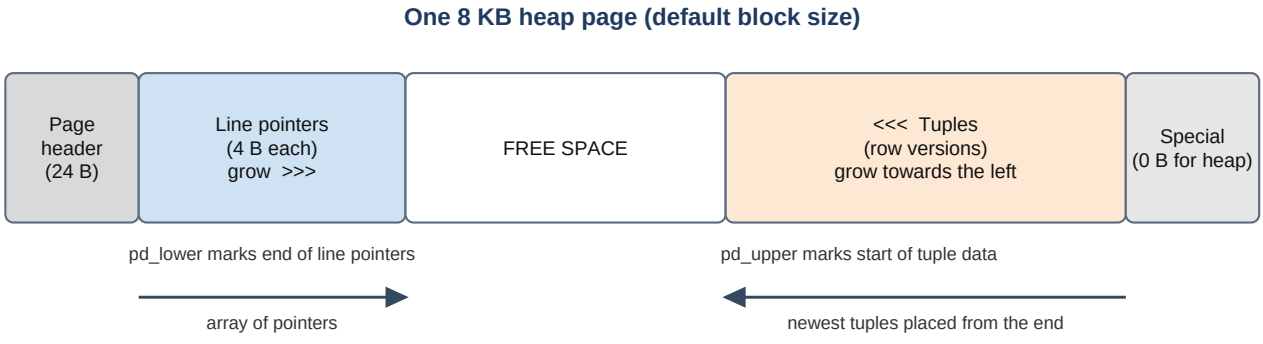
6.3 How a table is stored on disk

- Each table and index is stored in its own file named by its **relfilenode** number, inside the database folder, for example `base/16384/16397`.
- Files are split into **1 GB segments**: 16397, 16397.1, 16397.2 and so on.
- Besides the main file, a table can have **_fsm** (free space map: where free room exists in each page) and **_vm** (visibility map: which pages contain only rows visible to everyone). Vacuum and index-only scans use the visibility map.
- Large values are stored out-of-line using **TOAST** (see 6.6).

```
SELECT pg_relation_filepath('orders');      -- e.g. base/16384/16397
SELECT pg_size_pretty(pg_relation_size('orders'));
SELECT oid, datname FROM pg_database;
```

6.4 The page (block) layout

All table and index data is organised in fixed-size **8 KB pages**. PostgreSQL always reads and writes whole pages. A normal table (called a **heap**) page looks like Figure 3.



A tuple address is a ctid = (page number, line pointer number), for example (0,3)

Figure 3: Layout of one 8 KB heap page

- **Page header (24 bytes):** the LSN of the last WAL record that changed the page, checksum, flags, and pointers pd_lower, pd_upper and pd_special that mark the boundaries of the areas.

- **Line pointers (item IDs):** a growing array of 4-byte entries, each pointing to a tuple within the page. Using an indirection lets tuples be moved inside a page without changing their address.
- **Free space:** the gap between the two growing areas. When it is too small for a new tuple, a different page is used.
- **Tuples:** the actual row versions, filled from the end of the page backwards.
- **Special space:** unused in heap pages; used by index pages to hold index-specific data.

A row is addressed by its **ctid**, the pair (page number, line pointer number). Indexes store ctids to find rows. When an UPDATE changes no indexed column and the new version fits on the same page, PostgreSQL can use a **HOT (heap-only tuple)** update, which avoids touching the indexes. Setting a lower table `fillfactor` leaves room for this.

6.5 The tuple header

Each tuple carries a header of about 23 bytes in front of the column data. Its most important fields are:

- `xmin`: the ID of the transaction that created this row version.
- `xmax`: the ID of the transaction that deleted or updated it (zero if it is still current).
- `cmin/cmax`: command numbers inside a transaction.
- `ctid`: location of this version, or of the newer version if it was updated.
- **infomask** flags (including hint bits that cache whether xmin and xmax committed) and a **null bitmap**.

```
SELECT ctid, xmin, xmax, * FROM orders LIMIT 3;
```

6.6 TOAST

A row must fit in a page, but values can be much larger. **TOAST** (The Oversized-Attribute Storage Technique) handles this automatically. When a row is larger than about 2 KB, big column values are first compressed and, if still large, moved to a separate hidden table (`pg_toast.pg_toast_`) in chunks. The main row keeps a small pointer. A single value can be up to 1 GB. Queries that do not touch the large column never read the TOAST data.

6.7 Indexes

Indexes are separate files made of pages as well. They map column values to ctids of heap tuples. PostgreSQL offers several types:

Type	Best for
B-tree (default)	Equality and range queries, sorting, most workloads.
Hash	Simple equality lookups.
GiST	Geometric data, ranges, nearest-neighbour search, full-text search.
SP-GiST	Partitioned search structures such as tries and quadtrees.
GIN	Values containing many items: arrays, JSONB, full-text search.
BRIN	Very large tables where values are naturally ordered on disk (timestamps, IDs); tiny index size.

Because MVCC keeps old row versions, indexes also contain entries for dead tuples until vacuum cleans them.

6.8 Tablespaces

A **tablespace** is a directory on the file system where relation files can be placed, for example to keep hot tables on fast SSDs. Two exist by default: `pg_default` (the base/ directory) and `pg_global` (the global/ directory).

7. MVCC: how concurrent users are handled

Multi-Version Concurrency Control (MVCC) is how PostgreSQL lets many users read and write at the same time. The core rule is: **readers never block writers and writers never block readers**. This works because rows are never changed in place; instead, new versions are created.

7.1 How it works

- Every transaction that modifies data receives a **transaction ID (XID)**.
- **INSERT:** creates a tuple with xmin = the inserting XID.
- **DELETE:** does not remove the tuple; it sets xmax = the deleting XID.
- **UPDATE:** is a delete plus an insert. The old tuple gets xmax set, and a new tuple with the new values and a new xmin is added.
- At the start of a statement (Read Committed) or transaction (Repeatable Read) a backend takes a **snapshot**: the list of transactions that are still running. A tuple is visible if its xmin committed before the snapshot and its xmax is not set or has not committed as of the snapshot.
- Whether a transaction committed or aborted is looked up in the commit log (pg_xact). The result is cached in the tuple as **hint bits** to avoid repeated lookups.
- **ROLLBACK** costs almost nothing: the transaction is marked aborted and its tuples are simply never visible.

Time	Session A	Session B
t1	BEGIN; SELECT balance FROM acct WHERE id=1; returns 100	(idle)
t2	(idle)	UPDATE acct SET balance=150 WHERE id=1; COMMIT; creates a new tuple version
t3	SELECT balance ... Read Committed: sees 150. Repeatable Read: still sees 100 (its snapshot is unchanged).	(idle)

7.2 Isolation levels

Level	Behaviour
Read Committed (default)	Each statement sees data committed before it started. Two statements in the same transaction may see different data.
Repeatable Read	The whole transaction uses one snapshot taken at its first statement. Concurrent update conflicts raise a serialization error to retry.
Serializable	Strongest level. Uses Serializable Snapshot Isolation to guarantee results equal to some one-at-a-time order; may abort transactions that would break this, so applications must retry.

PostgreSQL accepts READ UNCOMMITTED but treats it as Read Committed; dirty reads never occur.

7.3 The price of MVCC: dead tuples and XID wraparound

- Old row versions remain until no running transaction can need them. These **dead tuples** take space and cause bloat; **vacuum** removes them (section 9).
- XIDs are 32-bit numbers, compared in a circular way. After roughly 2 billion transactions an old row would appear to be 'in the future'. To prevent this, vacuum **freezes** old tuples (marks them as visible to all). Autovacuum starts anti-wraparound vacuums when a table's age reaches `autovacuum_freeze_max_age` (default 200 million). Ignoring this can force PostgreSQL to stop accepting writes.

8. WAL, checkpoints and crash recovery

8.1 Write-Ahead Logging

Writing a modified data page to its place in a data file on every commit would be slow (random I/O) and unsafe (a crash could leave a half-written page). Instead PostgreSQL follows the WAL rule: **a change must be recorded in the log on durable storage before the corresponding data page can be written**.

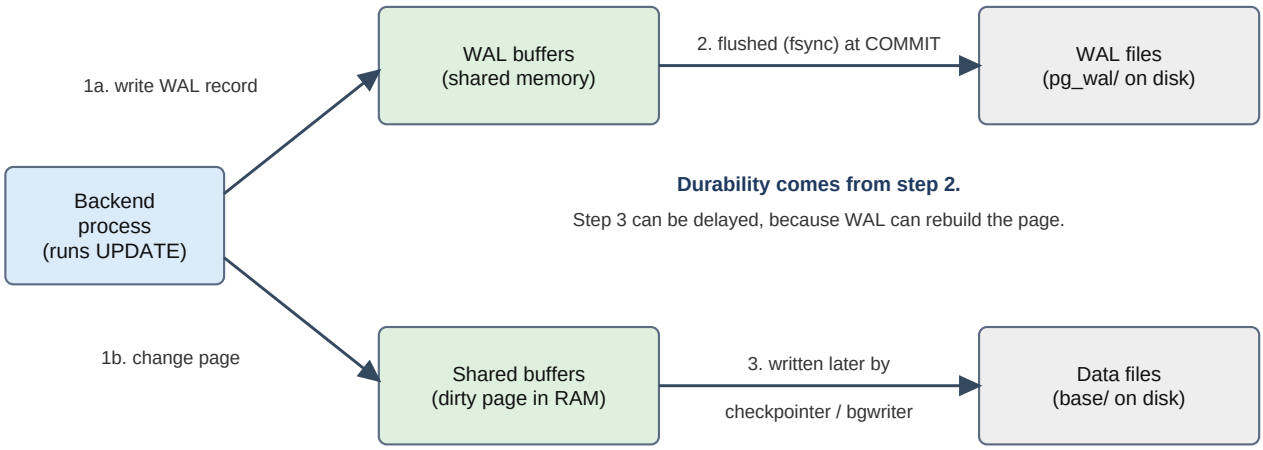


Figure 4: Write path: WAL first, data files later

- **1a. WAL record:** the backend creates a WAL record describing the change and places it in the WAL buffers.
- **1b. Page change:** the backend modifies the data page in shared buffers and marks it dirty, stamping the page header with the record's LSN.
- **2. Commit:** at COMMIT, the WAL up to the commit record is flushed (`fsync`) to the WAL file. Only after this is the client told 'committed'. WAL is written sequentially, which is fast.
- **3. Later:** the background writer and checkpointer write dirty pages to the data files at their own pace. A page is never written before the WAL up to its LSN is on disk.

WAL is stored in `pg_wal/` as **16 MB segment files**. Old segments are recycled or removed after a checkpoint, unless they are needed for archiving or replication.

8.2 Checkpoints

A **checkpoint** forces all dirty pages that existed at its start to be written to the data files, then writes a checkpoint record to WAL and updates `pg_control`. The position where recovery would need to start is called the **redo point**. Checkpoints happen when `checkpoint_timeout` (5 min) passes or WAL volume reaches `max_wal_size`, and are spread over time (`checkpoint_completion_target`) to avoid I/O spikes.

Full-page writes: the first time a page is modified after a checkpoint, the entire page image is put in WAL (`full_page_writes = on`). This protects against **torn pages**, where a crash occurs while an 8 KB page is only partly written to disk.

8.3 Crash recovery

- After an unclean shutdown, the startup process reads `pg_control` to find the last checkpoint's redo point.
- It then replays all WAL records from that point forward, reapplying changes to data pages (skipping pages whose LSN shows they already contain the change).
- When the end of WAL is reached, the database is consistent and opens for connections. Committed work is kept; uncommitted work is invisible.

8.4 Durability settings

Setting	Effect
fsync = on	Forces WAL and data to physical storage. Turning it off risks corruption after a crash; never disable in production.
synchronous_commit = on (default)	Commit waits until WAL is flushed to local disk (and to standbys when synchronous replication is configured).
synchronous_commit = off	Commit returns before WAL is flushed. A crash can lose the last fraction of a second of commits but does not corrupt data.
wal_level	minimal, replica (default) or logical; controls how much information is written to WAL for replication and decoding.

9. Vacuum and autovacuum

Because MVCC leaves dead tuples behind, a cleaning mechanism is essential. **VACUUM** does the following for a table:

- Finds dead tuples that no transaction can see any more and marks their space as reusable (also removes the matching index entries).
- Updates the free space map and the visibility map.
- Freezes old tuples to prevent transaction ID wraparound.
- Optionally truncates empty pages at the end of the file.

Plain VACUUM does not block normal reads and writes, but it does not usually shrink the file; the space is reused by later inserts. **VACUUM FULL** rewrites the table into a new compact file and gives space back to the operating system, but it holds an exclusive lock for the whole time. **ANALYZE** collects the planner statistics.

Autovacuum

The **autovacuum launcher** wakes periodically (every `autovacuum_naptime`, 1 min) and starts **workers** for tables that need work. A table is vacuumed when:

```
dead tuples > autovacuum_vacuum_threshold (50)
              + autovacuum_vacuum_scale_factor (0.2) x number_of_rows
```

So a table with 1,000,000 rows is vacuumed after about 200,050 dead tuples. For very large, busy tables this is often too late, so the factor is commonly lowered per table:

```
ALTER TABLE big_orders SET (autovacuum_vacuum_scale_factor = 0.02);
```

Long-running or idle-in-transaction sessions hold back the cleanup horizon and are a common cause of bloat. If bloat has already built up, tools such as pg_repack can remove it online.

10. Locking and concurrency

MVCC removes most conflicts, but locks are still needed to protect structure and to serialise conflicting writes. PostgreSQL uses several kinds:

- **Table-level (heavyweight) locks:** taken automatically by commands on tables, in eight modes from ACCESS SHARE (weakest) to ACCESS EXCLUSIVE (strongest). Held until the end of the transaction.
- **Row-level locks:** taken by UPDATE, DELETE and SELECT ... FOR UPDATE / FOR SHARE. They are recorded in the tuple header (xmax), not in memory, so locking millions of rows is cheap.
- **Lightweight locks (LWLocks) and spinlocks:** internal short-term locks protecting shared memory structures such as buffer mappings. Not visible to users.
- **Advisory locks:** application-defined locks managed via functions like pg_advisory_lock().

Operation	Table lock mode	Blocks
SELECT	ACCESS SHARE	Only ACCESS EXCLUSIVE
INSERT / UPDATE / DELETE	ROW EXCLUSIVE	Schema-changing and SHARE-type commands, but not other reads or writes
VACUUM (plain), ANALYZE, CREATE INDEX CONCURRENTLY	SHARE UPDATE EXCLUSIVE	Other maintenance and schema changes; not reads or writes
CREATE INDEX (non-concurrent)	SHARE	All writes to the table; reads still allowed
ALTER TABLE (most forms), DROP TABLE, TRUNCATE, VACUUM FULL, CLUSTER	ACCESS EXCLUSIVE	Everything, including SELECT

Deadlocks: if two transactions each wait for a lock the other holds, PostgreSQL detects it after `deadlock_timeout` (1 s) and aborts one of them with an error. To inspect waits:

```
SELECT pid, locktype, mode, granted, relation::regclass
FROM pg_locks WHERE NOT granted;

SELECT pid, pg_blocking_pids(pid) AS blocked_by, wait_event_type, wait_event, query
FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid)) > 0;
```

11. Replication, backup and recovery

11.1 Physical (streaming) replication

A **standby** server keeps an exact byte-level copy of the primary by continuously receiving and replaying its WAL. The primary's walsender streams WAL to the standby's walreceiver, and the startup process on the standby applies it.

- **Asynchronous** (default): the primary does not wait for the standby; very small lag, tiny risk of losing the latest commits if the primary fails.
- **Synchronous:** commit waits for one or more standbys to confirm, for zero data loss at the cost of latency (`synchronous_standby_names`).
- **Hot standby:** the standby accepts read-only queries, useful for scaling reads.
- **Replication slots** make the primary keep WAL until each standby has received it.

11.2 Logical replication

Instead of copying bytes, **logical replication** sends row changes (INSERT, UPDATE, DELETE) for chosen tables using a **publication** on the source and a **subscription** on the target. It works across major versions, allows replicating only some tables, and the target can have extra tables and indexes. It requires `wal_level = logical`.

11.3 Backup and point-in-time recovery (PITR)

- **Logical backup:** `pg_dump` / `pg_dumpall` export SQL or archive files; portable across versions, slower for large databases.
- **Physical backup:** `pg_basebackup` copies the data directory while the server runs.
- **WAL archiving** (archive_mode and archive_command) saves every completed WAL segment.
- **PITR:** restore a base backup, then replay archived WAL up to a chosen moment (`recovery_target_time`), for example just before an accidental DELETE.

Automatic failover and cluster management are not built in; they are provided by tools such as Patroni or repmgr, which monitor the primary and promote a standby (`pg_ctl promote`).

12. System catalogs and statistics

PostgreSQL stores its own metadata in ordinary tables called **system catalogs** (schema `pg_catalog`). The parser and planner consult them constantly. Useful ones:

Catalog / view	Information
pg_database, pg_roles	Databases and roles.
pg_namespace	Schemas.
pg_class	Every table, index, view and sequence (relation name, size estimates, relfilenode).
pg_attribute	Columns of every relation.
pg_index, pg_constraint	Index definitions and constraints.
pg_statistic / pg_stats	Planner statistics.
pg_settings	Current value and source of every configuration parameter.
information_schema	SQL-standard, portable views over the catalogs.

Cumulative statistics views show what is happening at runtime:

View	Use
pg_stat_activity	Current sessions, their state, running query, wait events.
pg_stat_user_tables / pg_stat_user_indexes	Scans, tuples read, inserts/updates/deletes, dead tuples, last vacuum and analyze.
pg_stat_database	Commits, rollbacks, cache hits, temp files per database.
pg_stat_bgwriter / pg_stat_checkpointer	Checkpoint and background writer activity (name depends on version).
pg_stat_replication	State and lag of each standby (on the primary).
pg_locks	All current locks.
pg_stat_statements (extension)	Execution statistics per normalised query; the best tool for finding slow or frequent queries.

13. Configuration and security files

postgresql.conf and postgresql.auto.conf

The main configuration file holds server parameters (memory, WAL, logging, replication, and so on). Changes made with `ALTER SYSTEM SET ...` are saved in `postgresql.auto.conf`, which overrides postgresql.conf. Parameters have different change levels:

- **postmaster** context: needs a restart (for example shared_buffers, max_connections, wal_level).
- **sighup** context: reload is enough (`SELECT pg_reload_conf();`).
- **user / superuser** context: can be changed per session with SET, or per role or database.

```
SELECT name, setting, unit, context, source FROM pg_settings WHERE name = 'work_mem';
ALTER SYSTEM SET work_mem = '16MB';
SELECT pg_reload_conf();
```

pg_hba.conf (host-based authentication)

Decides who may connect. Each line has: connection type, database, user, address and authentication method. Rules are checked from top to bottom and the first match wins.

```
# TYPE   DATABASE   USER      ADDRESS          METHOD
local    all        postgres                   peer
host     salesdb    appuser   10.0.0.0/24      scram-sha-256
hostssl  all        all       0.0.0.0/0        scram-sha-256
```

pg_ident.conf maps operating-system user names to database roles for peer or ident authentication. PostgreSQL's access control also uses **roles** (users and groups) with **privileges** (GRANT/REVOKE) on databases, schemas, tables and columns, and optional **row-level security** policies. Connections can be encrypted with SSL/TLS.

14. End-to-end walkthrough: one UPDATE from start to finish

This example ties all parts together. Suppose an application runs:

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 42;
COMMIT;
```

#	What happens	Component
1	The client connects; the postmaster forks a backend; authentication is checked against pg_hba.conf.	Postmaster, backend
2	The UPDATE text is parsed, analysed against the catalogs and planned (probably an index scan on id).	Parser, analyzer, planner
3	The transaction gets an XID. A ROW EXCLUSIVE lock is taken on the table; the row is located through the index and locked.	Lock manager, executor
4	The needed heap page is found in shared buffers (or read from disk into them).	Buffer manager
5	A new tuple version is written (HOT update if possible); the old version gets xmax = our XID. A WAL record is created in the WAL buffers. The page is marked dirty and its LSN updated.	Executor, WAL, shared buffers
6	COMMIT: a commit record is written and WAL is flushed to disk with fsync. The commit status is set in pg_xact. The client receives 'COMMIT'.	WAL writer path, CLOG
7	Other sessions now see the new version according to their snapshots; older snapshots still see the old version.	MVCC
8	Later, the bgwriter or checkpointer writes the dirty page to the data file.	Background processes
9	When no snapshot needs the old tuple, autovacuum marks its space reusable.	Autovacuum
10	If the server crashed after step 6 but before step 8, startup recovery would replay the WAL and restore the change.	Startup process, WAL
11	The standby (if any) receives the WAL through walsender/walreceiver and replays it.	Replication

15. Summary cheat sheet and commands to explore your server

Layer	Component	One-line role
Process	Postmaster	Supervises everything, accepts connections, forks backends.
Process	Backend	Serves one client: parse, plan, execute.
Process	Checkpointer / bgwriter	Write dirty pages to data files.
Process	WAL writer / archiver	Flush WAL; copy WAL to archive.
Process	Autovacuum	Clean dead tuples, update statistics, prevent wraparound.
Process	Walsender / walreceiver	Stream WAL to standbys.
Memory	Shared buffers	Cache of 8 KB data pages.
Memory	WAL buffers	Staging area for WAL records.
Memory	work_mem	Per-operation memory for sorts and hashes.
Storage	Heap / index files	Tables and indexes in 8 KB pages, 1 GB segments.
Storage	WAL (pg_wal)	Sequential change log; basis of durability, replication and PITR.
Storage	pg_control	Last checkpoint information for recovery.
Concept	MVCC	Row versions with xmin/xmax; readers do not block writers.
Concept	Checkpoint	Point after which older WAL is no longer needed for crash recovery.

Handy commands

```
SELECT version();
SHOW data_directory;   SHOW config_file;   SHOW hba_file;
SHOW shared_buffers;   SHOW work_mem;      SHOW max_connections;

-- sessions and background processes
SELECT pid, backend_type, state, wait_event FROM pg_stat_activity;

-- sizes
SELECT pg_size_pretty(pg_database_size(current_database()));
SELECT pg_size_pretty(pg_total_relation_size('orders'));

-- WAL position and checkpoint information
SELECT pg_current_wal_lsn();
SELECT * FROM pg_control_checkpoint();

-- cache hit ratio per database
SELECT datname, blks_hit::float / nullif(blks_hit + blks_read,0) AS hit_ratio
FROM pg_stat_database;

-- dead tuples and last autovacuum
SELECT relname, n_dead_tup, last_autovacuum FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC LIMIT 10;
```

Official documentation: `https://www.postgresql.org/docs/current/`. The chapters 'Internals', 'Server Administration' and 'Reliability and the Write-Ahead Log' go deeper into every topic in this guide.